UNIVERSIDADE VEIGA DE ALMEIDA

Engenharia de Software

P60200096 - Paradigmas de Linguagens de Programação

ATIVIDADE AVALIATIVA FINAL

Problema 1 - Sistema de Notas e Médias de Alunos

Integrante	RA
Samuel de Oliveira Pinheiro Rogério	1260101643
Miguel da Silva Moura	1260113472
Isaac Gabriel Pereira Rocha	1260123852
Victor Hugo Barborino Moura de Vasconcellos	1260122456

Professor: Rogério Mandelli

Campus: Tijuca

Turma: 4169EGSN1A2_P1

Grupo: 18

Rio de Janeiro

2026

Resumo

Este trabalho apresenta três soluções para o Sistema de Notas e Médias de Alunos, cada uma construída segundo um paradigma de programação diferente. A versão estruturada foi desenvolvida em C, a versão orientada a objetos em Java e a versão declarativa/funcional em Python. As três implementações leem nomes e notas, calculam médias individuais, classificam cada aluno como aprovado ou reprovado, ordenam a listagem por nome e calculam a média geral, a maior nota e a menor nota da turma. Adotou-se média mínima 7,0, pois o enunciado não especifica outro valor.

Palavras-chave: paradigmas de programação; C; Java; Python; programação estruturada; orientação a objetos; programação funcional.

Sumário

• 1. Introdução teórica
• 2. Descrição do problema
• 3. Implementação estruturada em C
• 4. Implementação orientada a objetos em Java
• 5. Implementação declarativa/funcional em Python
• 6. Testes e resultados
• 7. Análise comparativa
• 8. Concorrência e evoluções futuras
• 9. Conclusão
• 10. Referências

1. Introdução teórica

Um paradigma de programação é uma forma de organizar o raciocínio e expressar a solução de um problema. Paradigmas distintos podem produzir programas com o mesmo resultado externo, porém com estruturas, mecanismos de abstração e formas de manutenção diferentes. Este trabalho compara os paradigmas estruturado, orientado a objetos e declarativo/funcional.

1.1 Paradigma estruturado

A programação estruturada organiza o programa por meio de sequência, seleção e repetição. Funções e procedimentos dividem responsabilidades, reduzem repetição e tornam o fluxo de execução explícito. Na solução em C, as tarefas de leitura, validação, cálculo, ordenação, apresentação e liberação de memória foram separadas em funções.

1.2 Paradigma orientado a objetos

A orientação a objetos modela o domínio por classes e objetos que combinam estado e comportamento. Encapsulamento protege dados internos; herança permite especialização; polimorfismo possibilita tratar objetos por tipos abstratos comuns; e exceções representam situações anormais. Na versão Java, Aluno herda de Pessoa, implementa a interface Avaliavel e controla suas notas por métodos públicos.

1.3 Paradigma declarativo/funcional

A programação funcional enfatiza a transformação de dados por funções, preferencialmente puras, e reduz alterações de estado. A versão Python usa tuplas imutáveis, funções puras, lambda, map, filter, reduce e recursão. A intenção de cada transformação fica expressa diretamente, enquanto a linguagem administra grande parte dos detalhes de iteração.

1.4 Linguagens e conceitos fundamentais

C possui tipagem estática, compilação nativa e gerenciamento explícito de memória. Java também possui tipagem estática, executa sobre a JVM e oferece coleta automática de lixo. Python tem tipagem dinâmica, execução por interpretador e estruturas de alto nível. Nos três casos, variáveis locais possuem escopo limitado às funções ou métodos. Os dados permanecem apenas em memória durante a execução; portanto, a solução atual não implementa persistência em arquivo ou banco de dados.

2. Descrição do problema

O problema escolhido é o Sistema de Notas e Médias de Alunos. Seu contexto é o acompanhamento acadêmico de uma turma, permitindo registrar uma quantidade variável de alunos e de avaliações por aluno. O objetivo é produzir informações individuais e coletivas de maneira clara e validada.

2.1 Requisitos funcionais

- Ler o nome e as notas de n alunos.
- Permitir uma quantidade configurável de notas por aluno.
- Validar nomes não vazios e notas no intervalo de 0 a 10.
- Calcular a média aritmética individual.
- Classificar como Aprovado quando a média for maior ou igual a 7,0; caso contrário, Reprovado.
- Exibir a listagem em ordem alfabética pelo nome.

- Calcular a média geral da turma.
- Identificar a maior e a menor nota registradas.

2.2 Decisões e limites

A quantidade de notas é igual para todos os alunos em uma execução. O programa funciona em terminal e não grava dados permanentemente. A média de aprovação foi centralizada na constante MEDIA_APROVACAO, facilitando uma futura alteração da regra.

3. Implementação estruturada em C

A solução em C usa uma estrutura Aluno para agrupar nome, vetor de notas, quantidade de notas e média. O fluxo principal lê as dimensões, reserva memória, cadastra os alunos, calcula as médias, ordena com qsort, exibe o relatório e libera toda a memória alocada.

3.1 Modularização

- lerInteiroPositivo, lerNota e lerNome: leitura e validação.
- calcularMedia e calcularMediaGeral: cálculos numéricos.
- determinarSituacao: aplicação da regra de aprovação.
- compararAlunosPorNome: critério usado pelo qsort.
- encontrarMaiorEMenorNota: análise de todas as avaliações.
- exibirRelatorio: formatação da saída.
- liberarMemoria: prevenção de vazamento de memória.

3.2 Trecho representativo

```
double calcularMedia(const double notas[], int quantidade) {
    double soma = 0.0;
    for (int i = 0; i < quantidade; i++) {
        soma += notas[i];
    }
    return soma / quantidade;
}

const char *determinarSituacao(double media) {
    return media >= MEDIA_APROVACAO ? "Aprovado" : "Reprovado";
}
```

A implementação evidencia as estruturas de controle e o gerenciamento manual de memória. Essa transparência favorece o estudo do funcionamento de baixo nível, mas exige atenção adicional a ponteiros e liberações.

4. Implementação orientada a objetos em Java

A solução Java distribui responsabilidades entre classes. Pessoa é uma classe abstrata que encapsula o nome. Aluno especializa Pessoa, armazena notas e implementa Avaliavel. SistemaNotas mantém a coleção e calcula os indicadores. Main coordena entrada e saída. NotaInvalidaException representa uma regra de domínio violada.

4.1 Aplicação dos conceitos de OO

- Classes e objetos: cada aluno é uma instância com estado próprio.
- Encapsulamento: atributos são privados e as notas são expostas como lista não modificável.

- Herança: Aluno herda nome e contrato de resumo de Pessoa.
- Polimorfismo: Aluno é tratado como Pessoa ao gerar o resumo e como Avaliavel para cálculo e situação.
- Exceções: notas fora de 0 a 10 geram NotaInvalidaException; erros de entrada são tratados.

4.2 Trecho representativo

```
public class Aluno extends Pessoa implements Avaliavel {
    private final List<Double> notas;

    public void adicionarNota(double nota)
            throws NotaInvalidaException {
        if (nota < 0.0 || nota > 10.0) {
            throw new NotaInvalidaException(
                "A nota deve estar entre 0 e 10."
            );
        }
        notas.add(nota);
    }
}
```

O desenho facilita extensões como novos tipos de pessoa, diferentes regras de avaliação ou serviços de persistência. Em contrapartida, há mais arquivos e elementos arquiteturais do que nas demais versões.

5. Implementação declarativa/funcional em Python

A versão funcional representa cada aluno por NamedTuple e suas notas por tupla. As funções de cálculo recebem dados e retornam novos valores, sem alterar os argumentos. map transforma alunos em resultados, filter seleciona aprovados, reduce agrega valores e a recursão realiza leituras e somas.

5.1 Recursos funcionais empregados

- Funções puras: calcular_media, determinar_situacao, criar_resultado e outras não modificam estado externo.
- Imutabilidade: alunos, notas e resultados são representados por tuplas.
- map: converte alunos em registros de resultado.
- filter: seleciona os alunos aprovados.
- reduce: soma médias, reúne notas e obtém extremos.
- lambda: define pequenas operações de transformação e comparação.
- Recursão: soma valores e constrói sequências de alunos e notas.

5.2 Trecho representativo

```
def calcular_media_geral(resultados):
    medias = tuple(map(lambda resultado: resultado[1], resultados))
    return reduce(
        lambda total, media: total + media,
        medias,
        0.0,
    ) / len(medias)

aprovados = tuple(
    filter(lambda resultado: resultado[2] == "Aprovado", resultados)
)
```

A solução é compacta e expressiva. A principal limitação didática é que recursões muito profundas podem atingir o limite do interpretador Python; para o volume esperado nesta atividade, isso não

representa problema.

6. Testes e resultados

As três versões foram testadas com o mesmo conjunto de dados. O cenário possui três alunos e duas notas por aluno.

Aluno	Nota 1	Nota 2	Média	Situação
Carlos	8,0	6,0	7,0	Aprovado
Ana	9,0	10,0	9,5	Aprovado
Bruno	4,0	5,0	4,5	Reprovado

Resultado esperado: listagem Ana, Bruno e Carlos; média geral 7,00; maior nota 10,00; menor nota 4,00. Também foram verificados nomes vazios, quantidades não positivas, entradas não numéricas e notas fora do intervalo. C foi compilado com GCC 16.1, Java com OpenJDK 21 e Python foi validado no interpretador 3.12.

7. Análise comparativa

Critério	C estruturado	Java OO	Python funcional
Legibilidade	Fluxo explícito; mais detalhes técnicos.	Alta separação por classes.	Alta concisão nas transformações.
Modularização	Funções e struct.	Classes, interface e exceção.	Funções pequenas e composição.
Reutilização	Funções reutilizáveis.	Muito favorecida por abstrações.	Favorecida por funções genéricas.
Manutenção	Exige cuidado com memória.	Boa para sistemas maiores.	Boa quando o fluxo de dados é claro.
Desempenho	Compilação nativa e baixo overhead.	Bom desempenho na JVM.	Suficiente, com maior overhead.
Produtividade	Mais código operacional.	Mais estrutura inicial.	Implementação mais rápida e curta.
Tipos	Estáticos e explícitos.	Estáticos e orientados a classes.	Dinâmicos, com type hints.
Memória	Alocação e liberação manuais.	Coleta automática de lixo.	Gerenciamento automático.
Persistência	Não implementada.	Não implementada.	Não implementada.

Para este problema pequeno, Python oferece a implementação mais concisa. C torna o funcionamento interno mais visível. Java apresenta a melhor base para crescimento do sistema, pois suas classes permitem incorporar novas regras, formas de armazenamento e tipos de usuário com menor impacto nas partes existentes.

8. Concorrência e evoluções futuras

A implementação concorrente não é necessária para o cenário atual, pois a entrada ocorre sequencialmente em um único terminal. Em uma evolução para múltiplas turmas ou usuários, requisições poderiam ser processadas em paralelo. Nesse caso seria necessário proteger dados compartilhados, evitar condições de corrida e usar transações ao persistir informações.

- Persistir alunos e notas em arquivo ou banco de dados.
- Criar interface gráfica ou aplicação web.
- Permitir pesos diferentes para cada avaliação.
- Gerar boletins e exportar relatórios.
- Adicionar autenticação e acesso simultâneo por professores.
- Criar testes automatizados unitários e de integração.

9. Conclusão

As três soluções atenderam aos requisitos do Problema 1 e demonstraram que o mesmo comportamento pode ser obtido por organizações de código bastante diferentes. A abordagem estruturada foi direta e transparente; a orientação a objetos destacou encapsulamento, herança, polimorfismo e tratamento de exceções; e a abordagem funcional enfatizou imutabilidade e composição de transformações. Para uma solução acadêmica curta, C e Python apresentam menor complexidade estrutural. Para uma aplicação que deverá crescer, Java se mostrou o paradigma mais adequado por favorecer separação de responsabilidades, manutenção e extensão.

As principais dificuldades foram manter o mesmo conjunto de regras em três linguagens, validar entradas e representar adequadamente os conceitos de cada paradigma. O trabalho consolidou conhecimentos sobre tipos, escopo, memória, modularização, abstração e comportamento das linguagens.

10. Referências

- DEITEL, Paul; DEITEL, Harvey. C: Como Programar. São Paulo: Pearson.
- DEITEL, Paul; DEITEL, Harvey. Java: Como Programar. São Paulo: Pearson.
- SEBESTA, Robert W. Conceitos de Linguagens de Programação. Porto Alegre: Bookman.
- PYTHON SOFTWARE FOUNDATION. Python 3 Documentation. Disponível em: https://docs.python.org/3/.
- ORACLE. Java Documentation. Disponível em: https://docs.oracle.com/en/java/.
- FREE SOFTWARE FOUNDATION. GNU C Language Manual. Disponível em: https://www.gnu.org/software/c-intro-and-ref/manual/.